CS699 - Data Mining | Spring 2026

Intermediate Project Report

Predicting Disability Status in American Households

Dataset: 2023 American Housing Survey (AHS)
Source: U.S. Census Bureau

Team Members

Kunj Patel | Jai Sharma

February 25, 2026

1. Dataset Overview

1.1 About the Dataset

The dataset used in this project comes from the 2023 American Housing Survey (AHS), conducted by the U.S. Census Bureau. It is one of the largest national housing surveys in the United States and captures detailed information about housing units including physical condition, appliances, neighborhood quality, household income, and demographics of the people living there.

The working file is project_data.csv. Each row represents one household in America. The goal is to predict whether that household has at least one person with a disability.

1.2 Basic Structure

When we loaded the dataset, the first thing we checked was its basic dimensions:

```
df = pd.read_csv('project_data.csv')
print(df.shape)    # output: (9241, 145)
```

Property	Value
Total rows (households)	9,241
Total columns	145
Feature variables	144
Target variable	Class (Yes = has disability, No = no disability)

1.3 Target Variable: Class

The Class column is what we are predicting. Yes means at least one person in the household has a disability such as hearing, visual, mobility, or cognitive. No means no disability was reported.

We checked the full class distribution including missing values:

```
print(df['Class'].value_counts(dropna=False))
```

Class Value	Count	Note
No (no disability)	6,153	76.6% of labeled rows
Yes (has disability)	1,882	23.4% of labeled rows
Missing (NaN)	1,206	13.1% of all rows - must be removed
Total	**9,241**	

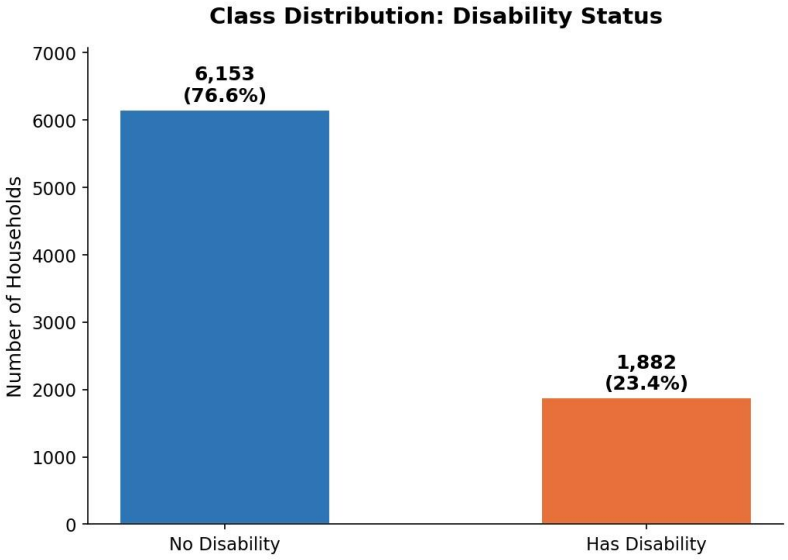

Figure 1: Class distribution - clear imbalance at 76.6% No vs 23.4% Yes

Two things stand out. First, 1,206 rows have no label and must be removed since supervised learning requires a known answer for every training example. Second, there is a significant class imbalance: 76.6% No vs 23.4% Yes. This will need to be addressed before model building to avoid the model being biased toward always predicting No.

2. Data Exploration

2.1 Missing Value Analysis

We ran a full missing value analysis across the entire raw dataset before any preprocessing:

```
total_missing = df.isnull().sum().sum()
print(round(total_missing / df.size * 100, 2), '% missing')
```

Metric	Value
Total cells in dataset	1,339,945
Total missing cells	123,509
Overall missing percentage	9.22%
Columns with at least 1 missing value	110 out of 145
Rows with at least 1 missing value	4,799 out of 9,241

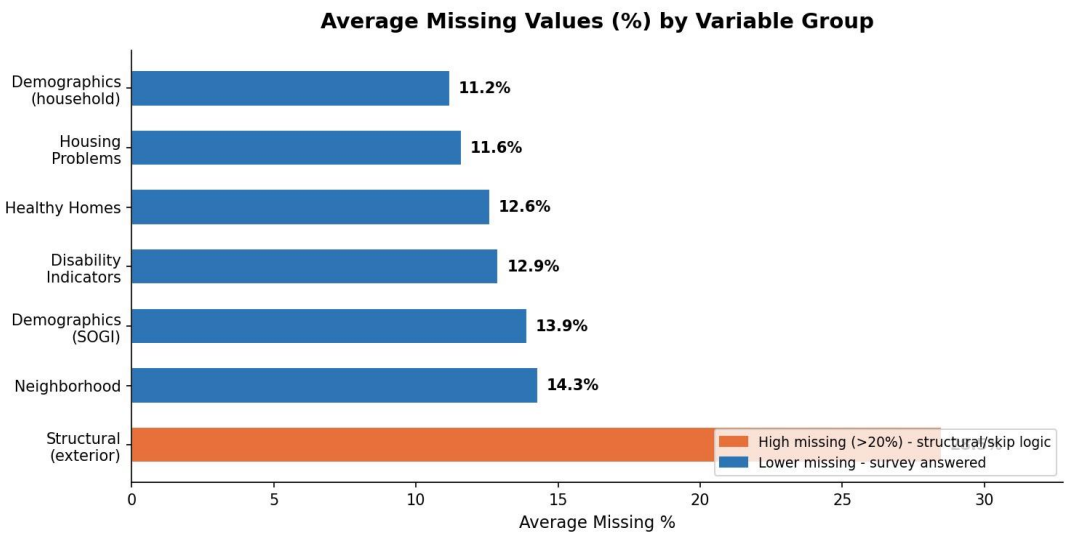

Figure 2: Average missing values by topic group - structural exterior variables highest due to survey skip logic

The structural exterior group has the highest missing rate. Questions about roof condition, foundation type, and exterior walls simply do not apply to apartment tenants, so those questions are automatically skipped in the survey. This is called survey skip logic - the missingness is not random, it is structural and expected.

2.2 Discovery: The 1,038 Vacant Unit Pattern

While exploring the data, we noticed something unexpected: exactly 40 columns all had the same missing count of 1,038. We checked if these were the exact same rows:

```
cols_1038 = missing_counts[missing_counts == 1038].index.tolist()
```

```
all_same  = all(df[col].isnull().equals(df[cols_1038[0]].isnull())
                for col in cols_1038)
print(all_same)    # output: True
```

Confirmed: all 40 columns had missing values in the exact same 1,038 rows. We then looked at what these rows actually contain:

Check	Result
Class label in all 1,038 rows	NaN - no label at all
Average columns filled in these 1,038 rows	54.9 out of 145
Average columns filled in normal rows	141.3 out of 145

These rows have no class label and only 54.9 out of 145 columns filled - compared to 141.3 for normal rows. This strongly indicates they are vacant housing units where no one lives. There is no household to interview so most survey questions are blank.

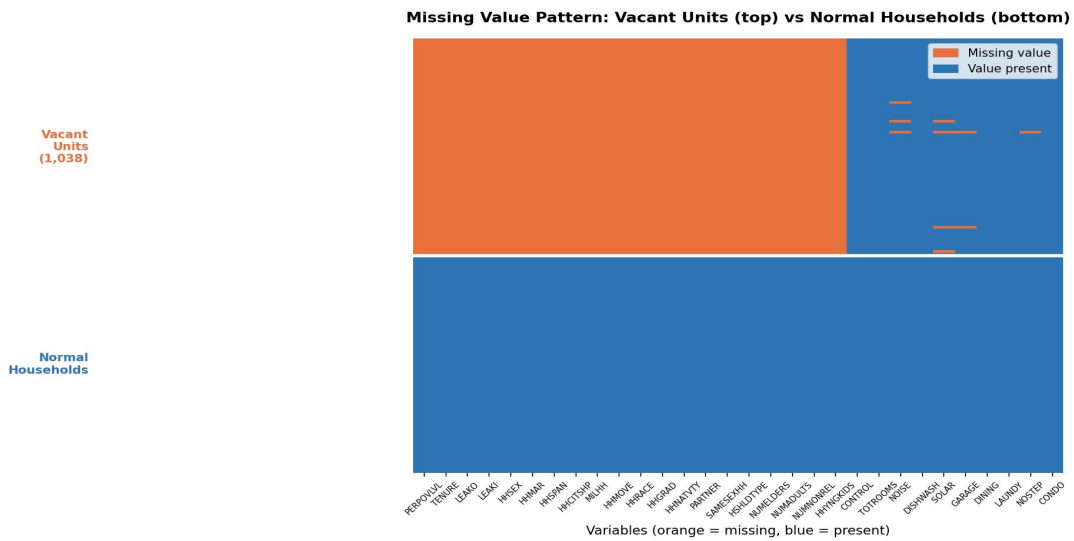

Figure 3: Missing value heatmap - vacant units (top, mostly orange) vs normal households (bottom, mostly blue)

The heatmap makes the pattern immediately obvious. The top block (vacant units) is almost entirely orange - missing. The bottom block (normal households) is almost entirely blue - present. This is not random missingness, it is a systematic structural pattern. No preprocessing can or should try to fill in the demographic data for a house where nobody lives.

2.3 Breaking Down the 1,206 Missing Class Rows

Group	Count	Reason
Missing Class AND all demographics	1,038	Vacant housing units
Missing Class but have household data	168	Interview not completed
Total missing Class	**1,206**	All removed before preprocessing

Both groups were removed. Supervised classification cannot be performed without a known class label for each row. This follows the approach from Lecture 1: ignore observations when the class label is missing.

2.4 Descriptive Statistics and Skewness

We computed descriptive statistics for the most important numeric variables. This step was critical for choosing the right imputation strategy.

Descriptive Statistics: Key Numeric Variables

Variable	Mean	Median	Std Dev	Min	Max	Skewness	Distribution Shape
Household Income ($)	105,813.4	75,000.0	144,761.7	0	4,109,700	13.73	right skewed
Elderly Members (65+)	0.5	0.0	0.7	0	4	1.17	right skewed
Total Household Size	2.3	2.0	1.4	1	13	1.39	right skewed
Number of Adults	1.8	2.0	0.9	0	8	1.41	right skewed
Number of Veterans	0.1	0.0	0.3	0	2	2.61	right skewed
Poverty Level (%)	334.3	370.0	166.0	1	501	-0.49	approx normal
Total Housing Cost ($/yr)	1,661.8	1,317.0	1,834.4	0	74,250	16.09	right skewed

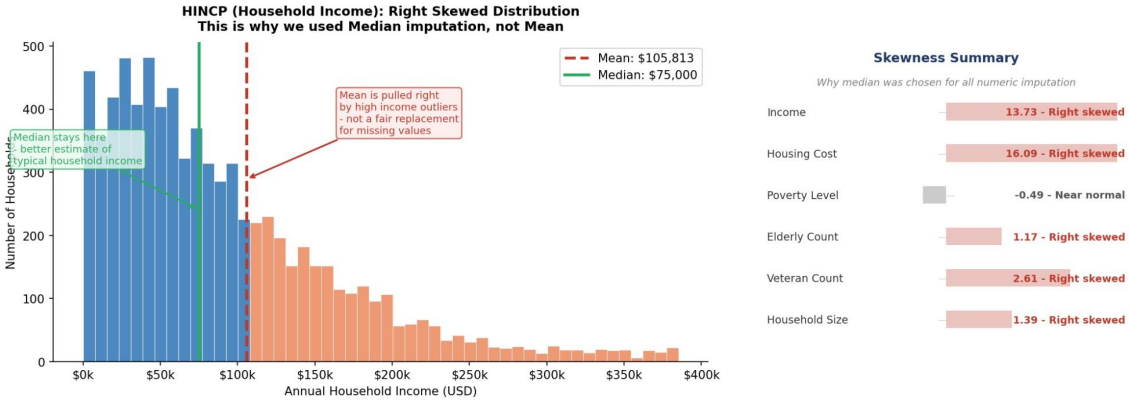

Figure 4: Descriptive statistics and skewness analysis for key numeric variables

The table shows that almost all numeric variables are right skewed. Household income (HINCP) has a skewness of 13.73. The mean is $105,813 but the median is $75,000. This $30,000 gap exists because a small number of very high income households pull the mean upward. Using the mean to fill in a missing income value would systematically overestimate income for typical households. The median is a more reliable replacement. This reasoning directly justifies our choice of median imputation for all numeric variables.

Total Housing Cost (TOTHCAMT) has an even higher skewness of 16.09. Number of veterans (NUMVETS) is at 2.61. Only the Poverty Level variable (PERPOVLVL) is approximately normal at -0.49. The conclusion is clear: median imputation is the right choice across the board.

2.5 Key Patterns Related to Disability

We explored which variables show the clearest relationship with the target class:

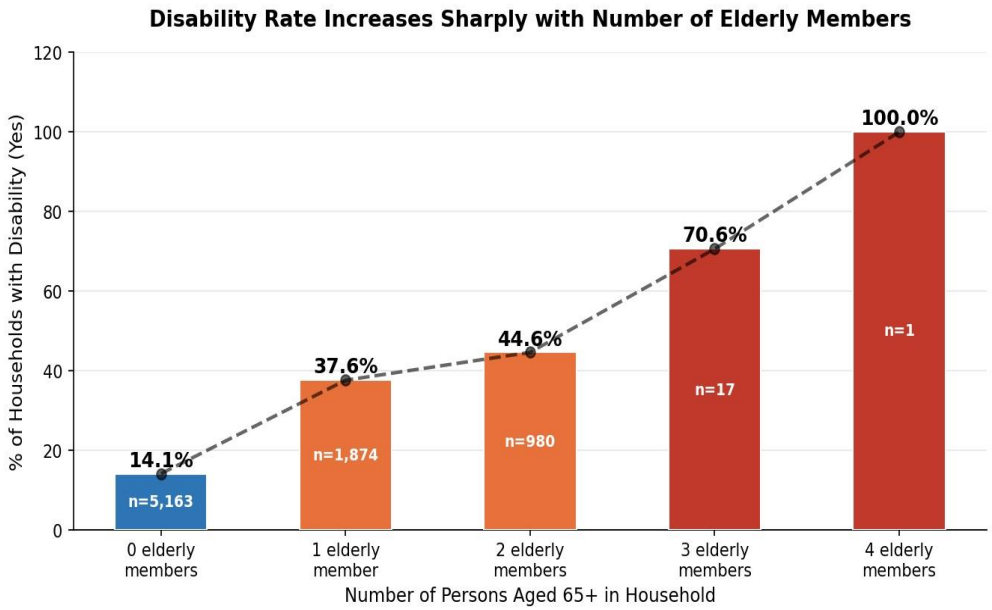

Figure 5: Disability rate increases sharply with number of elderly members - from 14.1% with no elderly to 100% with 4 elderly

This is the most informative chart from our exploration. It shows the actual probability of disability given the number of elderly household members. With 0 elderly members: 14.1% disability rate. With 1 elderly: 37.6%. With 2: 44.6%. With 3: 70.6%. The trend is consistent and strong. The variable NUMELDERS is very likely to be one of the strongest predictors in our model.

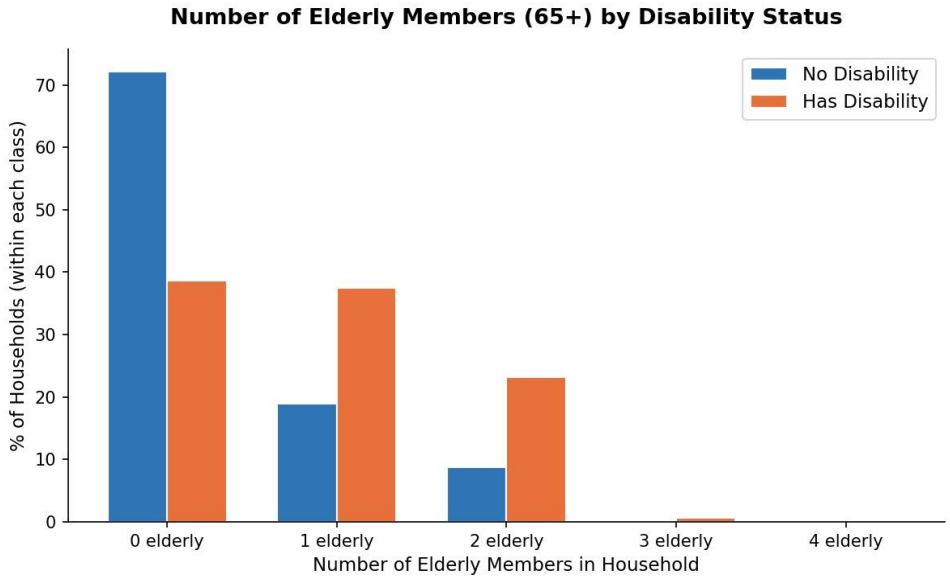

Figure 6: Distribution of elderly members by class - disability households have proportionally far more elderly members

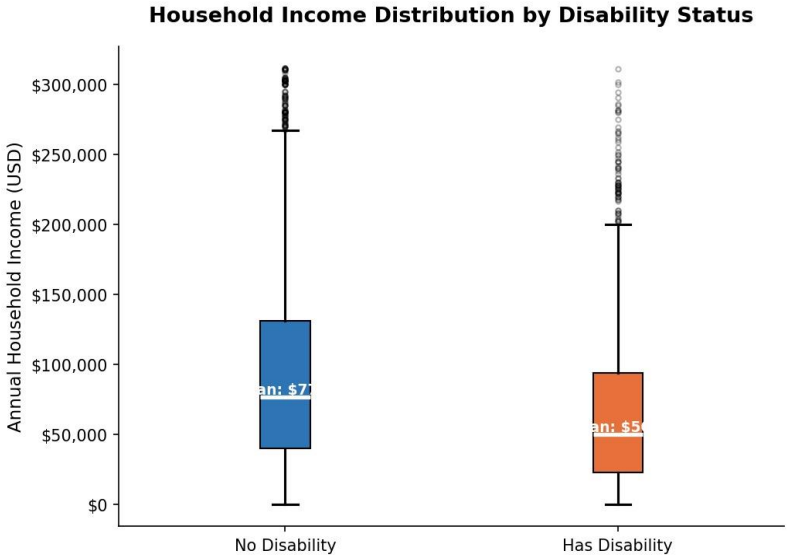

Figure 7: Income boxplot by class - disability households have lower median income ($50k) vs non-disability ($77k)

The income difference is notable. Disability households have a median income of around $50,000 compared to $77,000 for non-disability households. The IQR is also narrower for disability households, meaning less variation in income at the upper end. This makes intuitive sense: disability often reduces earning capacity. Income (HINCP) is likely to be another important predictor.

3. Codebook Analysis and Variable Classification

3.1 Why the Codebook Matters

When Python loads the dataset it reports 109 columns as float64, 35 as int64, and 1 as object (the Class column). At first glance this makes the dataset look almost entirely numeric. This is misleading.

After reading the AHS Dictionary Codebook (313 pages), we found that the majority of these columns are actually categorical variables stored as integer codes. For example: the variable NOISE has values 1 and 2. Without the codebook you might apply StandardScaler to it. But the codebook says 1 = Yes, noise is bothersome and 2 = No. These are labels, not quantities. Scaling them would be completely wrong and would introduce meaningless numerical relationships into the model.

This is why reading the codebook first was the most important step in the entire project. Without it, 72% of variables would have been preprocessed incorrectly - scaled or imputed with mean/median when they are actually categories.

3.2 How We Extracted Topics from the Codebook

We wrote Python code using the pdfplumber library to parse all 313 pages of the codebook PDF and automatically extract topic information for all 144 variables. Every variable entry in the codebook follows a consistent pattern:

```
Topic: Equipment and Appliances  Subtopic: Appliances  Availability: PUF
```

The code searched for each of the 144 variable names, located this line, and extracted the topic name. This automated approach saved time and avoided manual reading errors across hundreds of pages.

3.3 Variable Topics

Topic	Count	What it covers
Demographics	39	Age, race, education, veterans, household composition
Housing Problems	33	Leaks, mold, pests, broken walls, roof damage, electrical issues
Equipment and Appliances	22	Dryer, washer, AC, heating system, smoke alarms, solar panels
Structural	18	Rooms, stories, year built, lot size, foundation type
Housing Costs	11	Electricity, gas, water, trash, total monthly housing cost

Neighborhood	11	Crime ratings, school quality, transit access, noise, flood risk
Occupancy and Structure	3	Tenure (own/rent), condo status, HOA membership
Healthy Homes	3	Asthma diagnosis, number of asthma household members, ER visits
Administrative	2	Interview language, metro area code
Income	2	Total household income, total family income
Total	**144**	

3.4 Three Variable Types

After carefully reading the codebook for all 144 variables, we classified them into three types. Each type requires a completely different preprocessing approach.

Type 1 - Binary Categorical (55 variables):

These are Yes/No questions stored as 1 and 2. Example: NOISE (1 = Yes bothersome, 2 = No). The numbers are just labels with no mathematical meaning. These were recoded to 0 and 1 using label encoding.

Type 2 - Multi-Category Categorical (51 variables):

These have more than two category codes with no natural ordering between them. Example: DRYER (1 = Electric, 2 = Piped gas, 3 = LP gas, 4 = Other fuel, 5 = No dryer). Treating these as numbers would imply that No dryer (5) is five times Electric (1). This makes no sense. Following the dummy variable approach from Lecture 1, these were one-hot encoded.

Type 3 - True Numeric (37 variables):

These store real quantities where the number itself is meaningful. Example: NUMELDERS = number of persons aged 65+ in the household. A value of 2 truly means twice as many elderly members as a value of 1. These were standardized using z-score normalization.

Variable Type	Count	Preprocessing Applied
Binary Categorical	55	Mode imputation, label encode to 0/1
Multi-Category Categorical	51	Mode imputation, one-hot encode (dummy variables)
True Numeric	37	Median imputation, z-score standardization
Total	**143**	(144 features after KITCHENS and UNITSIZE reclassified)

Note: KITCHENS appeared in our initial numeric list but after checking the codebook it only had 2 unique values (0 and 1) meaning does the unit have a kitchen or not - so it is binary. Similarly

UNITSIZE had values 1-9 representing size categories (1 = less than 500 sq ft, 2 = 500-749 sq ft, etc.), not actual square footage - so it was moved to categorical.

4. Data Preprocessing

After completing the exploration, we performed all preprocessing steps in sequence. Starting point: raw dataset with 9,241 rows and 145 columns. Final preprocessed dataset: 8,035 rows and 747 columns.

Step 1: Drop CONTROL Column

CONTROL is a unique ID number assigned to each household for survey tracking. It has no predictive value at all - it is just a serial number. We dropped it immediately.

```
df = df.drop(columns=['CONTROL'])
# shape after: (9241, 144)
```

Step 2: Drop Rows with Missing Class Label

Following the approach from Lecture 1 - ignore observations when the class label is missing - we removed all 1,206 rows where Class is NaN. A supervised model cannot learn without a known answer for each training example.

```
df = df.dropna(subset=['Class'])
# shape after: (8035, 144)
# No: 6153 (76.6%),  Yes: 1882 (23.4%)
```

Step 3: Separate Features and Target

```
X = df.drop(columns=['Class'])    # (8035, 143)
y = df['Class']                   # (8035,)
```

Step 4: Identify Column Types

We defined column types based entirely on codebook knowledge, not Python data types. Python reports almost everything as numeric but as explained in Section 3, that is misleading. The actual breakdown is: 37 true numeric, 55 binary categorical, 51 multi-category categorical. These sum to 143 matching the feature count exactly.

Step 5: Impute Missing Values

After removing the 1,206 unlabeled rows, 25,861 missing values remained across the 143 feature columns. We used two strategies:

Numeric columns: median imputation. As the skewness analysis in Section 2.4 showed, most numeric variables are heavily right skewed. The mean gets pulled far from typical values by extreme cases. The median is more representative of where most households actually fall.

Categorical columns: mode imputation. We replaced missing values with the most frequently occurring category in each column.

```
num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')
X[numeric_cols]     = num_imputer.fit_transform(X[numeric_cols])
X[categorical_cols] = cat_imputer.fit_transform(X[categorical_cols])
X[binary_cols]      = cat_imputer.fit_transform(X[binary_cols])
# missing values remaining: 0
```

Step 6: Outlier Handling

We used the IQR method from Lecture 1 with factor 6. However, the standard row-removal approach caused a serious problem. With 39 numeric columns, a row gets removed if even one column has an outlier. Each column flags different households and the OR logic across all 39 columns removes over half the data regardless of what factor we use:

IQR Factor	Rows Removed	Rows Kept	Data Lost
1.5 (standard)	5,501	2,534	68.5%
3	4,787	3,248	59.6%
6 (lecture example)	4,482	3,553	55.8%
9	4,462	3,573	55.5%
15	4,439	3,596	55.2%

Going from factor 6 to factor 15 saves only 43 extra rows - the data loss stays around 55% no matter what. The professor's cirrhosis example in Lecture 1 used 8 numeric columns, where row removal works fine. Our dataset has 39 numeric columns - a fundamentally different situation. Removing an entire household because one variable out of 39 is extreme is not fair. We lose all the valid information that household provided about its rooms, appliances, neighborhood, and demographics.

Instead we used capping: replace any outlier value with the IQR boundary rather than deleting the whole row. For example a household income of $4,109,700 gets capped to $729,175 (Q3 + 6*IQR). The row stays. All 8,035 households are preserved. A total of 8,720 individual values were capped.

```
for col in numeric_cols:
    q1, q3  = X[col].quantile(0.25), X[col].quantile(0.75)
    iqr     = q3 - q1
    X[col]  = X[col].clip(lower=q1 - 6*iqr, upper=q3 + 6*iqr)
# rows kept: 8035,  outlier values capped: 8720
```

Step 7: Encode Target Variable

```
y = y.map({'Yes': 1, 'No': 0})
```

```
# Class 0 (No):  6153    Class 1 (Yes): 1882
# Imbalance ratio: 3.27:1
```

Step 8: Encode Feature Variables

Binary columns: Used LabelEncoder to convert 2-value columns to 0 and 1.

Multi-category columns: Used one-hot encoding (pd.get_dummies) to create dummy variables. Each category becomes its own 0/1 column. This follows the dummy variable approach shown in Lecture 1 and avoids implying any fake mathematical ordering between categories.

```
for col in binary_cols:
    X[col] = LabelEncoder().fit_transform(X[col].astype(str))
X = pd.get_dummies(X, columns=categorical_cols)
# shape after encoding: (8035, 747)
```

The 51 multi-category columns expanded into 604 dummy columns. Combined with 37 numeric and 55 binary, the final feature count is 747.

Step 9: Z-Score Standardization

We applied z-score standardization to all 37 numeric columns. Formula from Lecture 1: v' = (v - mean) / std. This brings all numeric variables to the same scale (mean = 0, std = 1). Without this, algorithms that use distance or gradients such as KNN or logistic regression would effectively ignore small-range variables like NUMELDERS (max = 4) because they are dominated by large-range variables like HINCP (max = $4 million).

```
scaler = StandardScaler()
X[numeric_cols] = scaler.fit_transform(X[numeric_cols])
```

4.1 Preprocessing Summary

Step	Before	After
1. Drop CONTROL	145 columns	144 columns
2. Drop missing Class rows	9,241 rows	8,035 rows
3. Impute missing values	25,861 missing values	0 missing values
4. Outlier capping (IQR x6)	8,720 extreme values	All capped, 8,035 rows kept
5. One-hot encoding	143 features	747 features
6. Z-score standardization	Raw numeric ranges	Mean = 0, Std = 1 for all numeric cols
Final dataset	9,241 x 145 (raw)	**8,035 x 747 (preprocessed)**

5. Class Imbalance and Next Steps

5.1 The Class Imbalance Problem

The preprocessed dataset has a class imbalance ratio of 3.27:1. For every household with disability, there are 3.27 without. This is a serious problem for classification.

Consider a naive model that always predicts No regardless of input. This model gets 76.6% overall accuracy - which sounds good. But its True Positive Rate for the Yes class is 0%. It never correctly identifies a single disability household. High accuracy but completely useless.

This is exactly why the assignment uses per-class TPR as the evaluation metric instead of overall accuracy. The minimum passing requirement is: Yes TPR >= 71% AND No TPR >= 67%. The naive model scores 0% for Yes TPR and immediately fails.

Model / Threshold	TPR - No Class	TPR - Yes Class
Naive model (predict all No)	100%	0% - completely useless
Minimum requirement (from assignment)	>= 67%	>= 71%
Extra credit target (from assignment)	>= 73%	>= 77%

Handling class imbalance is the most critical step before any model can be built. Without balanced training data, every model will learn to predict No because it is rewarded most of the time for doing so.

5.2 Next Steps for Final Report

The intermediate report covers data exploration and all preprocessing. The following steps are planned for the final report:

- Train/test split: partition the preprocessed dataset into training and test sets before any balancing is applied
- Class balancing: apply at least 4 balancing methods on the training set including at least one undersampling method as required by the assignment
- Feature selection: optionally reduce the 747 features to improve model efficiency and reduce noise
- Model building: apply at least 9 classification algorithms on each balanced dataset - giving at least 36 model combinations
- Parameter tuning: perform hyperparameter tuning for each algorithm using cross-validation
- Model evaluation: for each of the 36+ models report confusion matrix, TPR, FPR, precision, recall, F-measure, ROC area, MCC, and kappa
- Final model selection: identify the best performing model across all 36 combinations